From Matchings to Max-Flow Min-Cut

Daniel Mortensen

April 8, 2026

Contents

1 Matchings and Alternating Paths **2**
1.1 Bipartite Graphs . . . 2
1.2 Matchings . . . 2
1.3 Alternating and Augmenting Paths . . . 2
1.4 Finding Maximum Matchings via Augmenting Paths . . . 2

2 From Matchings to Network Flow **3**
2.1 Adding a Source and Sink . . . 3
2.2 Applications . . . 3

3 Formal Definitions: Flows and Cuts **3**
3.1 Flow Networks . . . 3
3.2 Flows . . . 4
3.3 Cuts . . . 4

4 Beyond Bipartite Graphs **4**

5 The Max-Flow Min-Cut Theorem **4**
5.1 Statement . . . 4
5.2 Residual Graph . . . 5
5.3 Proof of Weak Duality . . . 5
5.4 Proof of Strong Duality . . . 5

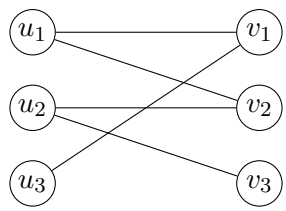
6 Algorithm and Examples **6**
6.1 The Ford–Fulkerson Algorithm . . . 6
6.2 Example . . . 6

7 Summary and Related Results **7**
7.1 Hall's Theorem . . . 7
7.2 Birkhoff–von Neumann Theorem . . . 7
7.3 Kőnig–Egerváry Theorem . . . 7
7.4 Menger's Theorem . . . 7
7.5 Unifying View . . . 8

1 Matchings and Alternating Paths

1.1 Bipartite Graphs

A **bipartite graph** $G = (U \cup V, E)$ has two disjoint vertex sets with edges only between them.



The fundamental question is: *how many independent pairs can we match?*

1.2 Matchings

A **matching** $M \subseteq E$ is a set of edges with no shared endpoints.

- A vertex is **matched** if it is an endpoint of some edge in M; otherwise it is **free**.
- A **maximum matching** has the largest possible $|M|$.
- A **perfect matching** matches every vertex ($|M| = |U| = |V|$).

Our goal is to find a maximum matching efficiently.

1.3 Alternating and Augmenting Paths

Given a matching M:

- An **alternating path** alternates between edges in M and edges not in M.
- An **augmenting path** is an alternating path that starts and ends at free (unmatched) vertices.

The key insight is that if an augmenting path exists, we can *flip* all edges along it—matched $\leftrightarrow$ unmatched—to obtain a matching of size $|M| + 1$.

Theorem 1 (Berge, 1957)**.** *A matching M is maximum if and only if there is no augmenting path with respect to M.*

1.4 Finding Maximum Matchings via Augmenting Paths

Berge's theorem immediately yields an algorithm:

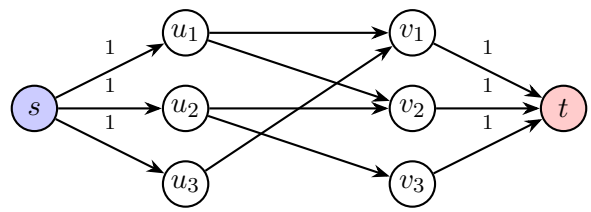
```
Start with any matching M (e.g. M = ∅)
repeat
    Search for an augmenting path P (BFS/DFS from free vertices)
    if P found then
        Augment: M ← M ⊕ P (symmetric difference)
    end if
until no augmenting path exists
return M (maximum matching)
```

Each augmentation increases $|M|$ by exactly 1, so the algorithm terminates after at most $|V|/2$ iterations.

2 From Matchings to Network Flow

2.1 Adding a Source and Sink

A bipartite matching problem is a network flow problem in disguise. Given a bipartite graph $G = (U \cup V, E)$, construct a flow network as follows:



1. Add a source s with directed edges to every vertex in U, each with capacity 1.
2. Add a sink t with directed edges from every vertex in V, each with capacity 1.
3. Direct all original edges from U to V with capacity 1.

In this construction, an integer maximum flow corresponds exactly to a maximum matching: the value of the max flow equals the size of the max matching.

2.2 Applications

Network flow models appear throughout combinatorics and applied mathematics:

- **Bipartite matching**: job assignments, course scheduling.
- **Transportation**: shipping goods through a network with link capacities.
- **Telecommunications**: routing data through links with bandwidth limits.
- **Image segmentation**: a min-cut separates foreground from background.
- **Baseball elimination**: determining whether a team can still win the league.
- **Component-to-line assignment**: assigning plot fragments to their underlying lines.

3 Formal Definitions: Flows and Cuts

3.1 Flow Networks

Definition 1 (Network)**.** *A **network** is a directed graph $G = (V, E)$ with a non-negative capacity function $c : E \to \mathbb{R}_{\geq 0}$, without multiple arcs. Without loss of generality, we may assume that if $(u, v) \in E$ then $(v, u) \in E$ as well; if $(v, u) \notin E$, we add it with $c(v, u) = 0$.*

Definition 2 (Flow network)**.** *If two nodes in G are distinguished—one as the **source** s and the other as the **sink** t—then (G, c, s, t) is called a **flow network**.*

3.2 Flows

Definition 3 (Flow)**.** *A **flow** on a flow network (G, c, s, t) is a function $f : E \to \mathbb{R}_{\geq 0}$ satisfying:*

1. ***Capacity constraint**: $0 \leq f(u, v) \leq c(u, v)$ for all $(u, v) \in E$.*
2. ***Flow conservation**: for all $v \in V \setminus \{s, t\}$:*

$$\sum_{u:(u,v)\in E} f(u, v) = \sum_{w:(v,w)\in E} f(v, w).$$

That is, flow into v equals flow out of v.

The **value** of a flow is the net flow leaving the source:

$$|f| = \sum_{v:(s,v)\in E} f(s, v) - \sum_{u:(u,s)\in E} f(u, s).$$

3.3 Cuts

Definition 4 (s-t cut)**.** *An s-t **cut** (S, T) is a partition of V into two sets $V = S \cup T$ with $s \in S$ and $t \in T$.*

The **capacity** of the cut is the total capacity of edges crossing from S to T:

$$c(S, T) = \sum_{\substack{u\in S,\, v\in T \\ (u,v)\in E}} c(u, v).$$

Intuitively, a cut represents a bottleneck: it is the maximum amount of flow that can cross from the s-side to the t-side.

4 Beyond Bipartite Graphs

Nothing in the definitions of flows and cuts requires a bipartite structure. The graph can have arbitrary topology—cycles, dense subgraphs, self-loops (with zero capacity), etc. The capacity constraints and flow conservation apply identically, and as we shall see, the max-flow min-cut theorem holds for *any* directed graph.

Bipartite matching is a special case: unit capacities on a layered graph. The general theory of network flow is strictly more powerful, handling real-valued capacities and arbitrary graph structure.

5 The Max-Flow Min-Cut Theorem

5.1 Statement

Theorem 2 (Max-flow min-cut; Ford–Fulkerson, 1956)**.** *In any flow network (G, c, s, t):*

$$\max_f |f| = \min_{(S,T)} c(S, T).$$

The maximum value of an s-t flow equals the minimum capacity of an s-t cut.

The following three conditions are equivalent:

1. f is a maximum flow.
2. There is no augmenting path in the residual graph G_f.
3. $|f| = c(S, T)$ for some cut (S, T).

5.2 Residual Graph

Given a flow f, the **residual graph** $G_f = (V, E_f)$ is defined as follows:

- For each edge $(u, v) \in E$ with $f(u, v) < c(u, v)$, include a **forward edge** (u, v) with residual capacity $c_f(u, v) = c(u, v) - f(u, v)$.
- For each edge $(u, v) \in E$ with $f(u, v) > 0$, include a **backward edge** (v, u) with residual capacity $c_f(v, u) = f(u, v)$.

An **augmenting path** is any $s \to t$ path in G_f. The backward edges are the key mechanism: they allow the algorithm to "undo" previously pushed flow, enabling rearrangements that no single forward push could achieve. This is the same idea as augmenting paths in matching.

5.3 Proof of Weak Duality

We first show that no flow can exceed any cut.

Claim: For any flow f and any s-t cut (S, T): $|f| \leq c(S, T)$.

Proof. By flow conservation, the net flow leaving S equals $|f|$:

$$\begin{aligned} |f| &= \sum_{\substack{u \in S,\, v \in T \\ (u,v) \in E}} f(u, v) - \sum_{\substack{u \in T,\, v \in S \\ (u,v) \in E}} f(u, v) \\ &\leq \sum_{\substack{u \in S,\, v \in T \\ (u,v) \in E}} f(u, v) \leq \sum_{\substack{u \in S,\, v \in T \\ (u,v) \in E}} c(u, v) = c(S, T). \end{aligned}$$

The first inequality drops the non-negative reverse-flow term; the second uses the capacity constraint $f(u, v) \leq c(u, v)$. □

5.4 Proof of Strong Duality

We now show that equality is achieved: there exist f^* and (S^*, T^*) with $|f^*| = c(S^*, T^*)$.

Proof. Run the Ford–Fulkerson algorithm: repeatedly find augmenting paths in G_f and push flow until no augmenting path exists. Let f^* be the resulting flow.

1. Since no augmenting $s \to t$ path exists in G_{f^*}, define:

$$S^* = \{v \in V : v \text{ is reachable from } s \text{ in } G_{f^*}\}, \qquad T^* = V \setminus S^*.$$

2. Since t is not reachable from s in G_{f^*}, we have $t \notin S^*$, so (S^*, T^*) is a valid s-t cut.
3. For every edge (u, v) with $u \in S^*$, $v \in T^*$: we must have $f^*(u, v) = c(u, v)$. If not, the forward edge (u, v) would exist in G_{f^*} with positive residual capacity, making v reachable from s—a contradiction with $v \in T^*$.

4. For every edge (v, u) with $v \in T^*$, $u \in S^*$: we must have $f^*(v, u) = 0$. If $f^*(v, u) > 0$, the backward edge (u, v) would exist in G_{f^*}, making v reachable—again a contradiction.

5. Therefore:

$$|f^*| = \sum_{u \in S^*,\, v \in T^*} f^*(u, v) - \sum_{v \in T^*,\, u \in S^*} f^*(v, u) = c(S^*, T^*) - 0 = c(S^*, T^*).$$

By weak duality, $|f^*| \leq c(S^*, T^*)$ for all cuts, and we have exhibited equality. Hence f^* is a maximum flow and (S^*, T^*) is a minimum cut. □

6 Algorithm and Examples

6.1 The Ford–Fulkerson Algorithm

Algorithm 1 Ford–Fulkerson method

```
Initialise f(u,v) ← 0 for all (u,v) ∈ E
while there exists an augmenting path P in G_f do
    δ ← min_{(u,v)∈P} c_f(u,v)                    ▷ bottleneck capacity
    for each edge (u,v) ∈ P do
        if (u,v) is a forward edge then
            f(u,v) ← f(u,v) + δ
        else                                      ▷ backward edge
            f(v,u) ← f(v,u) − δ
        end if
    end for
end while
return f
```

The **Edmonds–Karp** variant always chooses the shortest augmenting path (by BFS), giving a worst-case complexity of $O(VE^2)$.

6.2 Example

We trace the algorithm on a small network.

1. **Initialise**: set all flows to zero.

2. **Iteration 1**: find an augmenting path from s to t in the residual graph, compute the bottleneck capacity δ, and push δ units of flow along the path.

3. **Update**: recompute the residual graph (forward edges shrink, backward edges appear).

4. **Iterate**: repeat until no s-t path exists in the residual graph.

5. **Read off the min cut**: the set of vertices reachable from s in the final residual graph is S^*; the cut $(S^*, V \setminus S^*)$ is the minimum cut.

7 Summary and Related Results

The max-flow min-cut theorem is the cornerstone of a family of equivalent duality results in combinatorics. We state the four most important corollaries.

7.1 Hall's Theorem

Theorem 3 (Hall, 1935)**.** *A bipartite graph $G = (U \cup V, E)$ has a matching that saturates every vertex in U if and only if for every $S \subseteq U$:*

$$|N(S)| \geq |S|,$$

where $N(S)$ is the set of neighbours of S in V.

Connection to max-flow min-cut: Hall's condition is equivalent to saying that no s-t cut in the corresponding flow network has capacity less than $|U|$.

7.2 Birkhoff–von Neumann Theorem

Theorem 4 (Birkhoff, 1946; von Neumann, 1953)**.** *Every doubly stochastic matrix is a convex combination of permutation matrices.*

A **doubly stochastic matrix** is a non-negative matrix whose rows and columns each sum to 1. Equivalently, the set of doubly stochastic matrices is the convex hull of permutation matrices.

Connection to network flow: this theorem says that the LP relaxation of bipartite matching has no integrality gap—any fractional matching (doubly stochastic matrix) decomposes into a convex combination of integer matchings (permutation matrices).

7.3 Kőnig–Egerváry Theorem

Theorem 5 (Kőnig, 1931; Egerváry, 1931)**.** *In a bipartite graph, the size of a maximum matching equals the size of a minimum vertex cover.*

A **vertex cover** is a set of vertices that touches every edge. In general graphs, the minimum vertex cover can be much larger than the maximum matching, but in bipartite graphs LP duality gives exact equality.

Connection to network flow: the minimum vertex cover corresponds to a minimum cut in the associated flow network.

7.4 Menger's Theorem

Theorem 6 (Menger, 1927)**.** *The maximum number of vertex-disjoint s-t paths equals the minimum number of vertices (other than s, t) whose removal disconnects s from t.*

There is also an edge-disjoint version: the maximum number of edge-disjoint s-t paths equals the minimum number of edges whose removal disconnects s from t.

Connection to network flow: Menger's theorem is a direct corollary of max-flow min-cut with unit capacities.

7.5 Unifying View

Result	**Says**
Max-flow min-cut	max throughput = min bottleneck
Hall's theorem	perfect matching $\iff$ no small neighbourhood
Birkhoff–von Neumann	fractional matching = integer matching
Kőnig–Egerváry	max matching = min vertex cover
Menger's theorem	max disjoint paths = min separator

All of these results are facets of the same duality: **what you can push through a network equals what it takes to block it**.